



Retrieval Augmented Generation (RAG)

Building Enterprise AI Applications using Amazon Bedrock

Key Takeaway: RAG allows LLMs to answer questions using your organization's private knowledge — without retraining the model.

🎓 LEARNING OBJECTIVES

- Understand **why RAG is required** and the limitations of Large Language Models
- Master the **complete RAG architecture** — ingestion, chunking, embeddings, and retrieval
- Explore **enterprise AI-Ops** patterns and Amazon Bedrock Knowledge Bases

Why Do We Need RAG?

Large Language Models are trained on vast public data — but they are fundamentally blind to your organization's private knowledge. This gap between general intelligence and enterprise context is exactly what RAG solves.

LLMs Know

- Public documentation and standards
- Programming languages and frameworks
- Cloud, Kubernetes, and Linux concepts
- General engineering best practices

LLMs Do NOT Know

- Jira tickets & Slack discussions
- Confluence runbooks & RCA documents
- Terraform, Helm Charts, Kubernetes YAML
- GitHub history & app-specific documentation

📄 **Enterprise Problem:** Ask "Why is Payment Service crashing?" and the LLM responds: *"I don't know your company's infrastructure."* RAG fixes this by injecting your private knowledge into the LLM's context window at query time.



RAG vs. Fine-Tuning

Both RAG and fine-tuning extend LLM capabilities — but they solve fundamentally different problems. Choosing the wrong approach wastes time, money, and engineering effort.

Fine-Tuning

- Modifies model weights permanently
- Expensive to train and re-train
- Static — knowledge freezes at training time
- Best for **changing model behavior**

RAG

- Model weights remain unchanged
- Cost-effective — ingest new documents anytime
- Dynamic — knowledge updates immediately
- Best for **enterprise knowledge retrieval**

Real-world example: Your company generates 500 Jira tickets, 300 Slack messages, and 20 deployment reports every single day. Would you retrain the LLM daily? Absolutely not. With RAG, new documents are ingested into the Knowledge Base and become available immediately — no retraining required.

Complete RAG Architecture

RAG is a two-phase pipeline. The **offline ingestion phase** processes and indexes your enterprise documents. The **online runtime phase** retrieves relevant knowledge and generates grounded answers — all in milliseconds.

Offline — Ingestion Pipeline

01

Document Parsing

Extract text, tables, images from PDFs, Confluence, S3

02

Data Cleaning & Chunking

Split documents into semantically meaningful chunks

03

Metadata Creation

Tag chunks with source, environment, severity, and application

04

Embedding & Indexing

Convert chunks to vectors and store in Vector Database

Online — Runtime Pipeline

01

Query Embedding

Convert the user's question into a vector representation

02

Similarity Search

Find top-K most relevant chunks in the vector database

03

Reranking

Score and reorder retrieved chunks by true relevance

04

Prompt Augmentation & LLM

Inject chunks into prompt — LLM generates a grounded answer

❏ Amazon Bedrock automates parsing, chunking, embedding generation, indexing, and retrieval for fully managed Knowledge Bases.

Offline vs. Online Pipeline

Understanding the separation between the ingestion and runtime pipelines is essential for designing scalable RAG systems. One creates knowledge; the other retrieves it.

Offline — Ingestion

Runs once whenever new documents are added to the knowledge base.

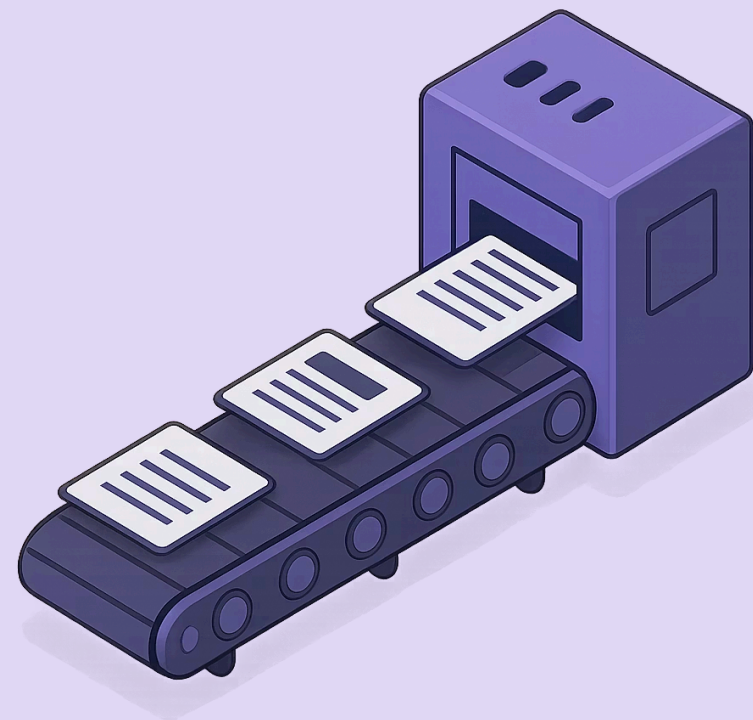
Processing is asynchronous and does not affect query latency.

- PDF / Slack / Jira / Confluence parsing
- Chunking and metadata tagging
- Embedding generation
- Vector index population

Online — Runtime

Runs synchronously for every user question. Latency matters — this pipeline must be fast, reliable, and accurate.

- Query embedding in real time
- Semantic similarity search
- Top document retrieval and reranking
- LLM prompt augmentation and response



Document Ingestion & Chunking Strategies

A 300-page PDF cannot become a single vector. Chunking splits documents into smaller, semantically coherent pieces that can be individually embedded, indexed, and retrieved. Choosing the right chunking strategy dramatically impacts retrieval quality.

	Fixed Size Splits by character count. Best for FAQs and short, uniform documents.
	Default / Recursive Splits by paragraphs and sentences. Best for general documentation and wikis.
	Semantic Groups sentences by meaning. Best for enterprise search where context matters most.
	Hierarchical Preserves document structure. Best for books, runbooks, and nested content.
	Structure-Aware Respects code and config syntax. Best for Terraform, YAML, and Helm Charts.
	Multimodal Handles images, audio, and video. Best for mixed-media enterprise content.

☐ **Chunk Overlap:** Overlapping content between adjacent chunks (e.g., "Password Expired") ensures context is never lost at chunk boundaries — critical for multi-sentence reasoning.

Embedding Models

An embedding model converts text into a high-dimensional numerical vector that encodes semantic meaning. Similar concepts produce similar vectors — enabling the vector database to find relevant content even when the exact words differ.

How Embeddings Work

Different words, same meaning → similar vectors:

"Payment Pod Failed"

[0.23, -0.55, 0.81, ...]

"Payment Service Crashed"

[0.22, -0.53, 0.79, ...]

Near-identical vectors confirm these phrases share the same semantic meaning — even though no keywords overlap.

Supported Embedding Models



Amazon Titan Text Embeddings V2

AWS-native, optimized for enterprise text retrieval



Cohere Embed

High-performance multilingual embeddings



Amazon Nova Multimodal

Embeds text, images, and video for multimodal search



Critical distinction: The Embedding Model handles search. The LLM handles reasoning. These are separate models with separate responsibilities.

Vector Databases & Semantic Search

A vector database stores embeddings, metadata, and document references — not the original PDFs. It enables lightning-fast similarity comparisons across millions of vectors to find the most semantically relevant chunks for any query.

Open Source Options

FAISS

Meta's high-speed in-memory index

ChromaDB

Developer-friendly local vector store

Qdrant

Production-ready with payload filtering

Weaviate

GraphQL-native semantic search engine

pgvector

Vector search inside PostgreSQL

AWS-Managed Options

OpenSearch Serverless

Fully managed, scalable vector search

Aurora PostgreSQL

pgvector inside managed RDS

Amazon S3 Vectors

Cost-optimized vector storage at scale

Neptune Analytics

Graph-enhanced vector retrieval

Semantic Search in practice: Instead of matching keywords, the system matches *meaning*. A query for "Database Password" will surface chunks containing "Database Credentials" or "Authentication Failure" — because their vectors are geometrically close in embedding space.

Retrieval, Metadata Filtering & Reranking

Retrieval quality is the single most important factor in RAG accuracy. Combining retrieval strategies, metadata filters, and reranking ensures the LLM receives only the most relevant, targeted context — reducing hallucination and improving answer precision.



Dense Retrieval

Uses embedding vectors and cosine similarity to find semantically similar chunks. Excellent for natural language queries.



Sparse Retrieval (BM25)

Matches exact keywords and terms. Best for acronyms, error codes, and highly specific technical terms.



Hybrid Retrieval

Combines dense and sparse signals for maximum enterprise accuracy. The recommended approach for production systems.



Metadata Filtering

Pre-filter by Application, Environment, Severity, or Team before vector search — dramatically narrowing the candidate pool.



Reranking

A cross-encoder model rescores the top 20 retrieved chunks and returns the top 5 most relevant — improving final answer quality.

📌 Amazon Bedrock supports dedicated reranking models to improve retrieved chunk relevance before they are sent to the LLM.



SITE RELIABILITY ENGINEER

Enterprise AI-Ops: RAG in Action

RAG transforms incident response by combining live operational data with retrieved historical knowledge — producing grounded, evidence-based answers instead of generic suggestions.

Live Investigation

Kubernetes Tools

Real-time pod status, events, and resource metrics

CloudWatch

Live logs, alarms, and metric streams

Prometheus

Current latency, error rate, and saturation data

Historical Investigation (RAG)

Jira & Slack

Past incident threads and team discussions

Runbooks & RCAs

Documented resolutions from prior incidents

Confluence

Architecture docs and on-call procedures



Current Alert

Payment Pod — **CrashLoopBackOff**



Retrieved: INC-1257

Root Cause: Database Password Expired

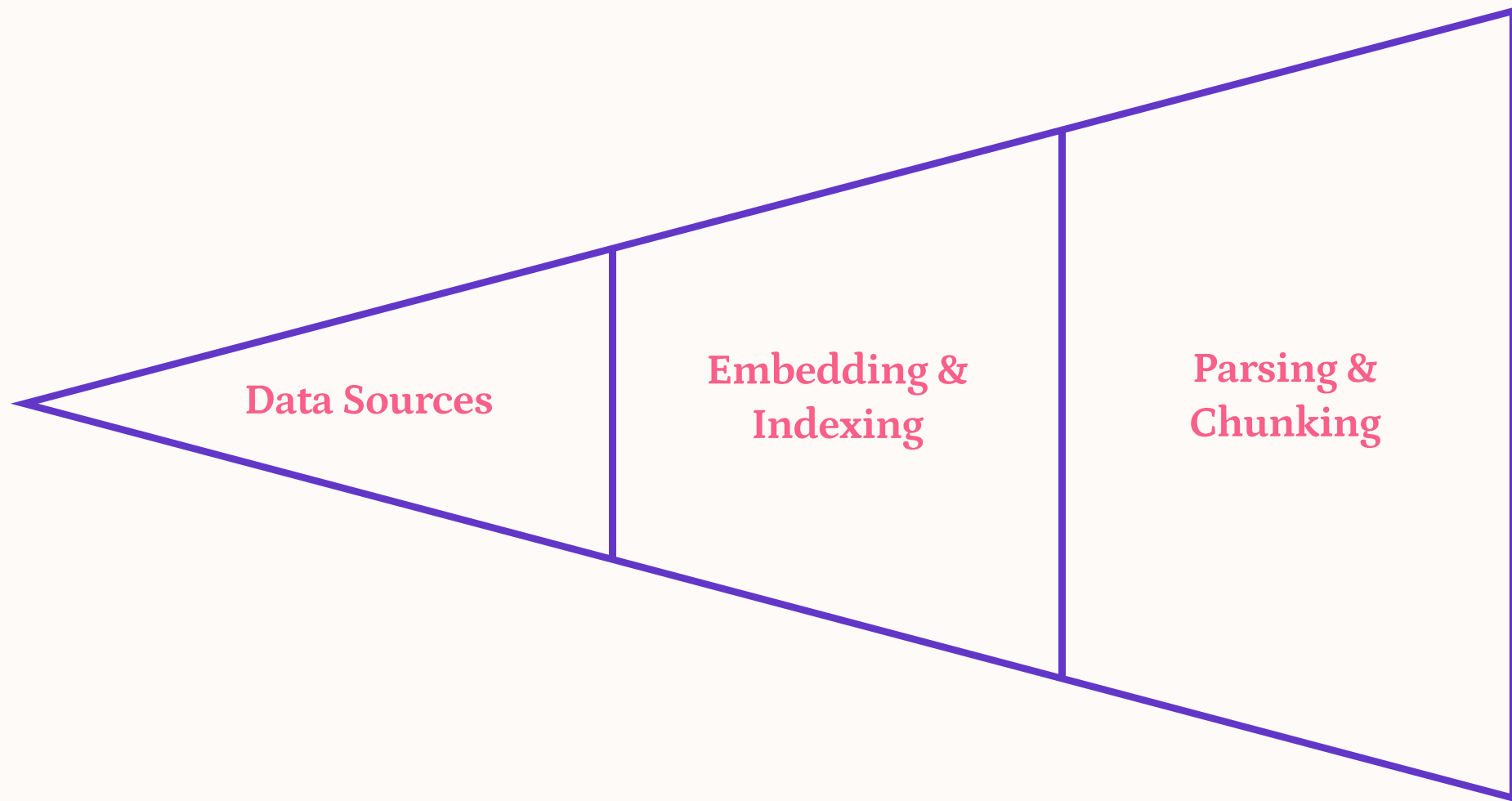


AI Resolution

Action: Restart Secret Sync Controller

Amazon Bedrock Knowledge Bases

Amazon Bedrock Knowledge Bases provides a fully managed RAG pipeline — eliminating the need to build and maintain custom ingestion, embedding, indexing, and retrieval infrastructure. Connect your enterprise data sources and let Bedrock handle the rest.



Multiple Data Source Connectors

S3, Confluence, SharePoint, Salesforce, and web crawlers — all natively supported with no custom ETL required.



Managed Ingestion & Embedding

Automatic document parsing, chunking strategy selection, embedding generation, and vector store population at scale.



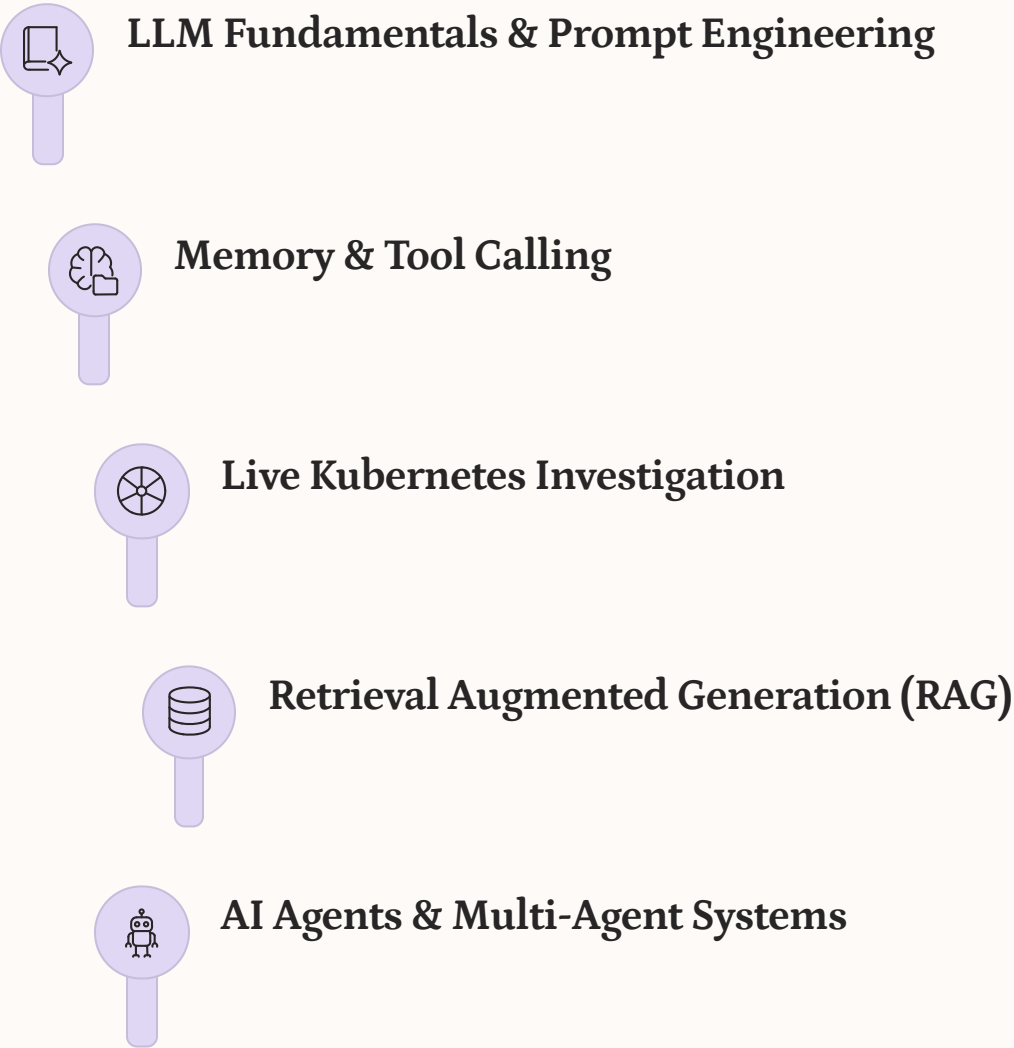
Source Citations & Multimodal

Every response includes source attribution. Supports text, images, and video through multimodal embeddings.

Enterprise AI Roadmap & Mental Model

RAG is a critical milestone in the enterprise AI journey — sitting between core LLM capabilities and fully autonomous AI agents. Understanding where each component fits helps you build incrementally toward a production-grade AI-Ops platform.

The Enterprise AI Roadmap



Component Responsibility Model

Component	Responsibility
LLM	Reasoning and answer generation
Embedding Model	Convert text and queries into vectors
Vector Database	Store embeddings and perform similarity search
Knowledge Base	Enterprise organizational memory
Tools	Retrieve live operational data
AI Agent	Plan and execute multi-step workflows

❑ **Core principle:** RAG gives your LLM a memory of everything your organization has ever documented — without touching the model weights.